

PHPUnit Application Architecture Test

Idea: write architecture tests as well as feature and unit tests. Protect your architecture code style!

Example

Don't use repositories in controllers use only in services classes. Take three layers "repositories", "services", "controllers" and add asserts on dependencies.

```
$controllers = $this->layer()->leaveByNameStart('App\\Controllers');
$services    = $this->layer()->leaveByNameStart('App\\Services');
$repositories = $this->layer()->leaveByNameStart('App\\Repositories');

$this->assertDoesNotDependOn($controllers, $repositories);
$this->assertDependOn($controllers, $services);
$this->assertDependOn($services, $repositories);
```

Installation

Install via composer

```
composer require --dev ta-tikoma/phpunit-architecture-test
```

Add trait to Test class

```
abstract class TestCase extends BaseTestCase
{
    use ArchitectureAsserts;
}
```

Use

- Create test
- Make layers of application
- Add asserts

```
public function test_make_layer_from_namespace()
{
    $app = $this->layer()->leaveByNameStart('PHPUnit\\Architecture');
    $tests = $this->layer()->leaveByNameStart('tests');

    $this->assertDoesNotDependOn($app, $tests);
    $this->assertDependOn($tests, $app);
}
```

Run

`./vendor/bin/phpunit`

Test files structure

- tests
 - Architecture
 - * SomeTest.php
 - Feature
 - Unit

How to build Layer

- `$this->layer()` take access to layer with all objects and filter for create your layer:
 - leave objects in layer only:
 - * `->leave($closure)` by closure
 - * `->leaveByPathStart($path)` by object path start
 - * `->leaveByNameStart($name)` by object name start
 - * `->leaveByNameRegex($name)` by object name regex
 - * `->leaveByType($name)` by object type
 - remove objects from layer:
 - * `->exclude($closure)` by closure
 - * `->excludeByPathStart($path)` by object path start
 - * `->excludeByNameStart($name)` by object name start
 - * `->excludeByNameRegex($name)` by object name regex
 - * `->excludeByType($name)` by object type
- you can create multiple layers with split:
 - `->split($closure)` by closure
 - `->splitByNameRegex($closure)` by object name

Asserts

Dependencies

Example: Controllers don't use Repositories only via Services

- `assertDependOn($A, $B)` Layer A must contains dependencies by layer B.
- `assertDoesNotDependOn($A, $B)` Layer A (or layers in array A) must not contains dependencies by layer B (or layers in array B).

Methods

- `assertIncomingsFrom($A, $B)` Layer A must contains arguments with types from Layer B
- `assertIncomingsNotFrom($A, $B)` Layer A must not contains arguments with types from Layer B

- `assertOutgoingFrom($A, $B)` Layer A must contains methods return types from Layer B
- `assertOutgoingNotFrom($A, $B)` Layer A must not contains methods return types from Layer B
- `assertMethodSizeLessThan($A, $SIZE)` Layer A must not contains methods with size less than SIZE

Properties

- `assertHasNotPublicProperties($A)` Objects in Layer A must not contains public properties

Essence

You can use `$layer->essence($path)` method for collect data from layer. For example get visibility of all properties in layer: `$visibilities = $layer->essence('properties.*.visibility');` .

- `assertEach($list, $check, $message)` - each item of list must passed tested by \$check-function
- `assertNotOne($list, $check, $message)` - not one item of list must not passed tested by \$check-function
- `assertAny($list, $check, $message)` - one or more item of list must not passed tested by \$check-function

Alternatives

- Deptrac
- PHP Architecture Tester
- PHPArch
- Arkitect

Advantages

- Dynamic creation of layers by regular expression (not need declare each module)
- Run along with the rest of tests from phpunit
- Asserts to method arguments and return types (for check dependent injection)
- Asserts to properties visibility